

Services, WorkManager, Build & release process

Android Lecture 5

Today's Agenda

- Services
- Work Manager
- Build process
- Release process
- Q&A



Services

Services

- Documentation
- Long running operation in background
- Not bound with UI
- Can expose API for other applications
- By default runs on UI thread
- **Types:** Foreground, Background, Started, Bound

Services

- Needs to be declared in manifest:



AndroidManifest.xml

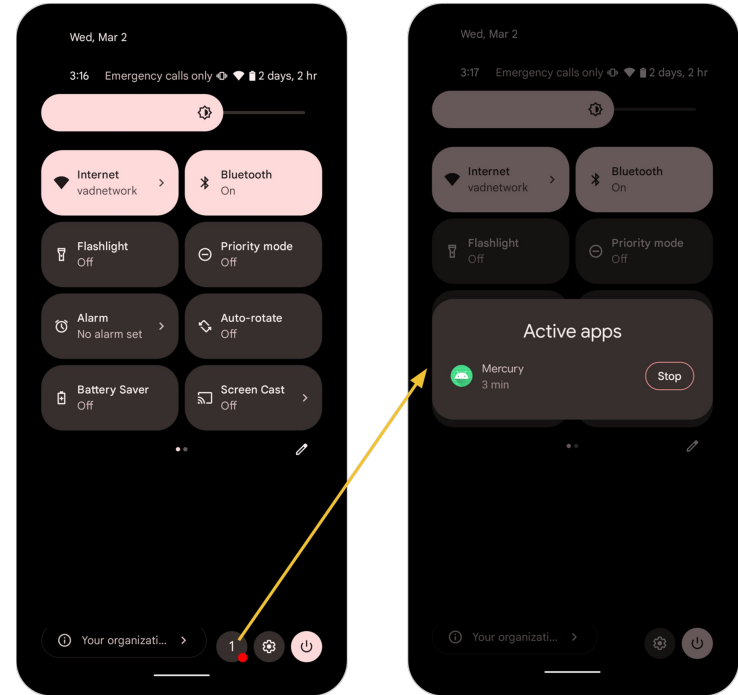
```
<manifest ... >
  ...
  <application ... >
    <service android:name=".ExampleService" />
    ...
  </application>
</manifest>
```

Background service

- On background by default
- Strongly limited since Android Oreo (API 26)
 - Not possible to start background service when app is not on the foreground
 - [Details in documentation](#)
- *Example usage*: Sync against backend
- Alternatively use WorkManager

Foreground service

- Service process has higher priority
- User is actively aware of it – requires **permanent notification**
 - Notification shown under Ongoing header
 - Lower than Android 13: notification cannot be dismissed
 - Android 13 and higher: notification dismissible by default
- System not likely to kill foreground services
- Apps targeting Android 9 (API 28) or higher must define
 - FOREGROUND_SERVICE permission (normal permission)
- *Example usage*: Downloading files
- Alternatively use foreground Workers



Started Service

Service that is started, runs independent of UI and does not return result to caller.

Example usage: started from Activity in order to save data to database.

Starting:

- Started by another component calling `startService()`
- When started, it's lifecycle is independent from the caller

Stopping:

- **System will not stop started service**
- Service can stop itself: `Service#stopSelf()`
- Other component can stop the service: `Context#stopService()`

Started Service



Starting of Service

```
context.startService(Intent(context, HelloService::class.java))
```



Must implement onStartCommand

```
class ExampleService : Service() {  
    ...  
  
    override fun onStartCommand(intent: Intent?, flags: Int, startId: Int): Int {  
        // The service is starting, due to a call to startService()  
        return startMode  
    }  
  
    ...  
}
```

Bound service

Service is bound when another component calls `bindService()` and offers client-server interface for communication.

Example usage: Music player service with controls in Activity.

Binding:

- Component calls `bindService()`
- Service overrides `Service#onBind(intent: Intent): IBinder?`
- `IBinder`: Object providing API to communicate with service

Unbinding:

- Client calls `unbindService()`

Bound service

Bound Service implementation

```
class LocalService : Service() {  
  
    private val binder = LocalBinder() // Binder given to clients.  
  
    /** Method for clients. */  
    val randomNumber: Int  
        get() = Random().nextInt(100) // Generate random number  
  
    /**  
     * Class used for the client Binder.  
     */  
    inner class LocalBinder : Binder() {  
        // Return this instance of LocalService so clients can call public methods.  
        fun getService(): LocalService = this@LocalService  
    }  
  
    override fun onBind(intent: Intent): IBinder {  
        return binder  
    }  
}
```

Bound service: ServiceConnection

Client needs to implement
ServiceConnection
callbacks

```
ServiceConnection implementation

private val connection = object : ServiceConnection {

    override fun onServiceConnected(className: ComponentName, service: IBinder) {
        // Connection with the service has been established.
        val binder = service as LocalService.LocalBinder
        val service = binder.getService()
        val randomNum = service.randomNumber
        ...
    }

    override fun onServiceDisconnected(arg0: ComponentName) {
        // Service was disconnected.
        // Process hosting service crashed or been killed
        // Service connection remain active (onServiceConnected can be called again)
    }

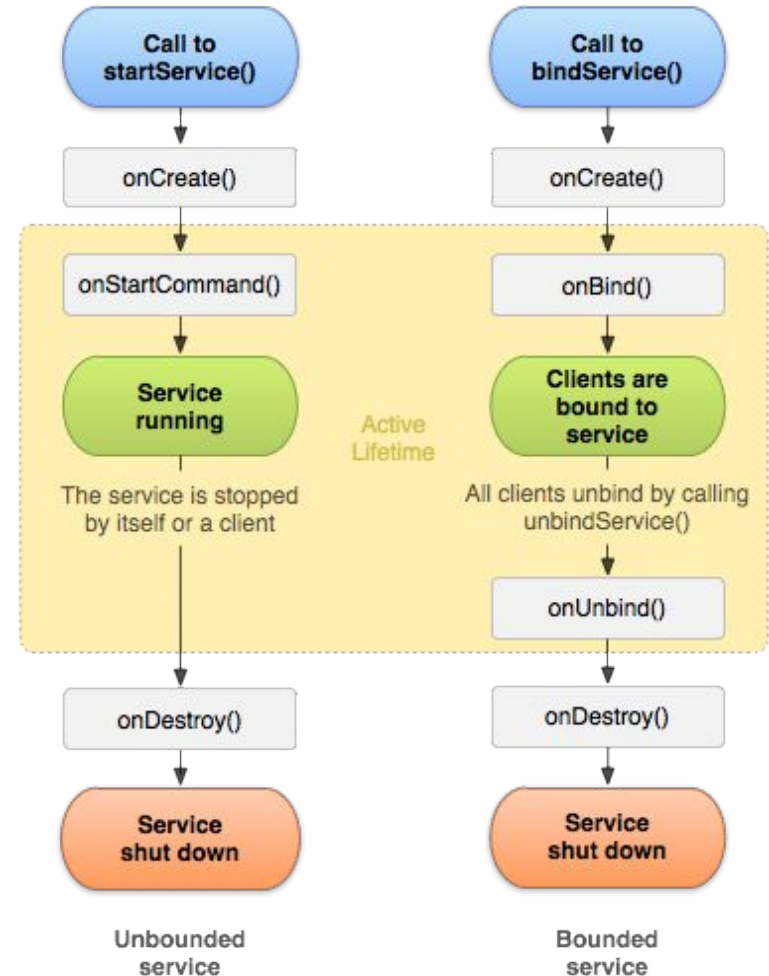
    override fun onBindingDied(name: ComponentName?) { // Optional impl
        // Binding is dead
        // Can happen during app update
        // Unbind and rebind
    }

    override fun onNullBinding(name: ComponentName?) { // Optional impl
        // Service#onBind returns null
        // Unbinding is still required
    }
}
```

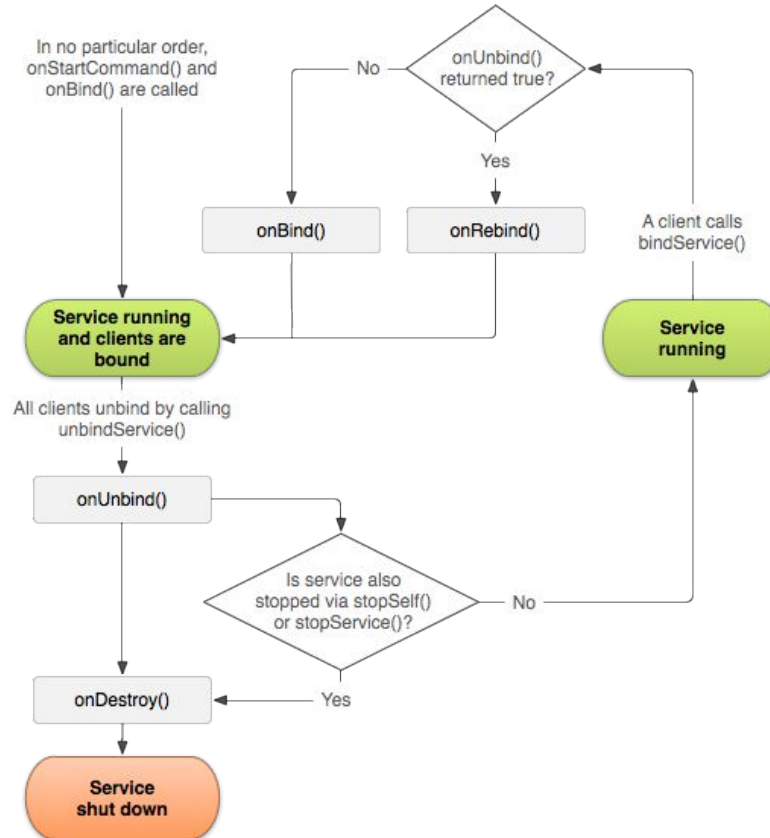
Lifecycle of Service

onStartCommand:

- Intent as parameter
- Return value specifies behaviour what to do when it is killed by system (not recreated, recreated with same intent, recreated with null intent)



Lifecycle of a Bound Service





WorkManager

WorkManager

- Documentation
- Backward compatible up to API 14
- Ensure execution even if app or device restarts
- Adheres to doze mode
- **Uses:**
 - JobScheduler on devices with API 23+
 - Combination of BroadcastReceiver + AlarmManager API 14-22

WorkManager

- One-off or periodic
- Monitor and manage scheduled tasks
- Chain tasks
- Observable status
- **Work constraints**
 - Network
 - Charging status
 - ...

Work requests



```
// Create a Constraints object that defines when the task should run•  
  
val constraints = Constraints.Builder()  
    .setRequiresDeviceIdle(true)  
    .setRequiresCharging(true)  
    .build()  
  
// ...then create a OneTimeWorkRequest that uses those constraints•  
  
val compressionWork = OneTimeWorkRequestBuilder<CompressWorker>()  
    .setConstraints(constraints)  
    .build()
```

Workers

```
class UploadWorker(appContext: Context, workerParams: WorkerParameters) : Worker(appContext, workerParams) {  
    override fun doWork(): Result {  
        // Get the input  
        val imageUriInput = getInputData().getString(Constants.KEY_IMAGE_URI)  
  
        // Do the work  
        val response = uploadFile(imageUriInput)  
  
        // Create the output of the work  
        val outputData = workDataOf(Constants.KEY_IMAGE_URL to response.imageUrl)  
  
        // Return the output  
        return Result.success(outputData)  
    }  
}
```

Enqueue Worker

- enqueue()
- enqueueUniqueWork()
- enqueueUniquePeriodicWork()



```
val workManager: WorkManager = WorkManager.getInstance(context)
workManager.enqueue(compressionWork)
```

Foreground Workers

- Long-running workers
- Can run longer than 10 minutes
- Need to show notification
- Uses foreground service

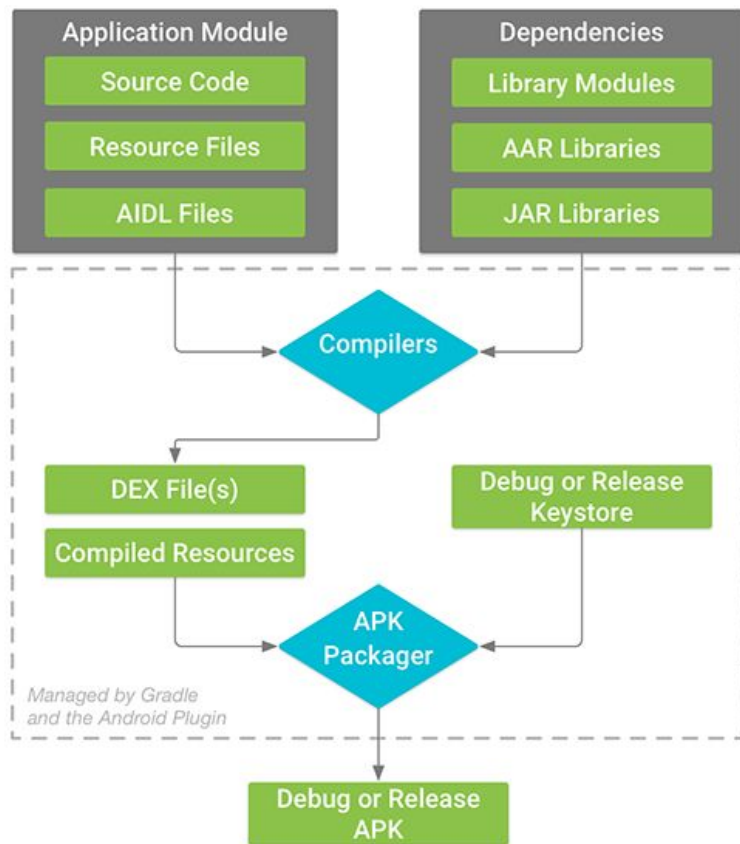
```
class UploadWorker(appContext: Context, workerParams: WorkerParameters) : Worker(appContext, workerParams) {  
    override fun doWork(): Result {  
        // Mark worker as long-running/important  
        setForeground(ForegroundInfo(notificationId, getNotification()))  
        ...  
        return Result.success()  
    }  
}
```



Build Process

Build process: Simplified

1. Kotlin and Java is compiled to **.class** files
2. KAPT/KSP processing: Hilt, Room, etc. annotations generate more **.class** files
3. All **.class** files are converted to **DEX** (Dalvik Executable) files (so that they be able to run on Android)
 - a. Multidex files - contain method references



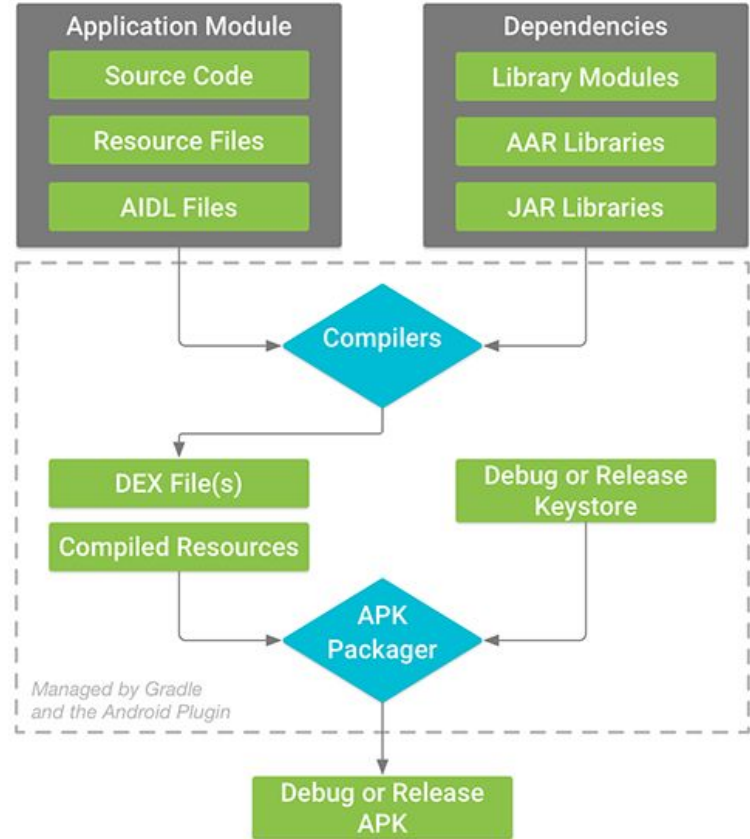
Build process: Simplified

4. Resource processing and packaging

- R.java class is generated by AAPT2 tool with ids of resources

5. Manifest merging from all modules and parts of app

6. Shrinking and obfuscation



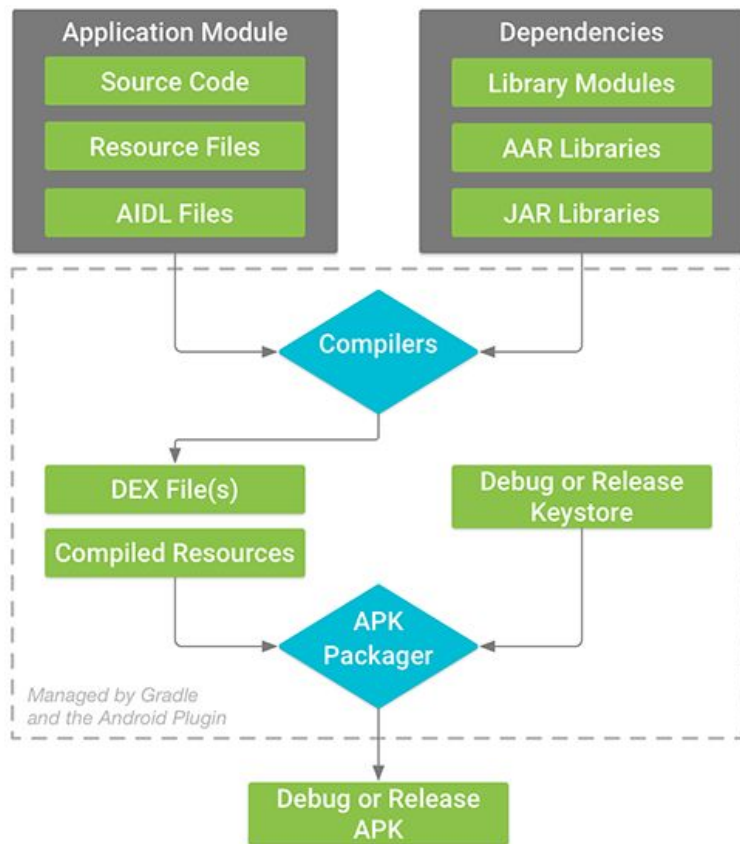
Build process: Simplified

7. Packaging: compiled code, resources, manifest, libraries, etc.

Result: **apk** or **aab** (app bundle)

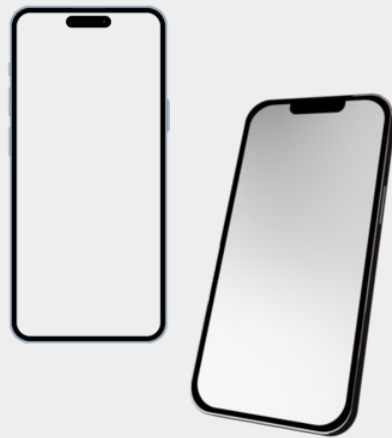
8. Signing of apk

- Debug builds = auto-signed.
- Release builds = your keystore that Gradle uses to sign apk



Bundle

- It is not directly installable
- contains everything needed to generate apk optimised for specific device
 - all app architectures
 - all screen densities resources
 - ...
- required by Google Play since 2021
- Benefits: smaller downloads for users, dynamic features (download only when needed)



Release Process

Before release

- Make sure debug logs are not present in the final build
- Define version of the application (only higher versions can be uploaded)
- Review permissions, unused resources/assets
- Make sure the app is aligned with Google policy
- Use obfuscation
- Ensure backend is ready & running (if needed)
- Build & sign bundle

Testing

Types:

- End-to-End / Smoke = testing of whole big flows, simulating standard user actions
- Regression = avoiding reintroducing previous bugs; sanity check of old logic
- Integration = integration of new components/features and how it works in a system
- Performance/load/stress = big data, high network traffic, many users, etc.

Automation:

- Instrumentation vs. Unit tests (Mockito, JUnit)
- Screenshot/Snapshot testing
- UI tests (frameworks: UiAutomator, Espresso)

Static code analysis:

- *tools*: detekt, lint, etc.
- maintenance of quality of code
- avoidance of common error
- unified code style

Continuous integration (CI)

- Building of apk/bundle
- Test Automation (running daily/nightly, after every new commit, manual trigger, etc.)
- Automation of release/deploy process
- Examples: Teamcity, Jenkins, Bitrise, Gitlab, ...

Google Play Release

- Developer account is needed (25 USD)
- Release types:
 - Internal testing
 - Alpha (open testing)
 - Beta
 - Production
- *Staged rollout* - start at small percentage, increase progressively
- Release notes - public vs. internal
- Google Play listing: screenshots, texts, pricing

Release monitoring

- Possibility of crashes, ANR, user complains
- Tools: Google Play Vitals, Firebase Crashlytics, other analytics and logging tools
- User review
- Backend traffic (load, changes in traffic)
- Number of users, version distribution, marketing statistics, etc.
- In case of problem -> **halt and hotfix**

Thank you for attention

Feel free to leave feedback about this lesson:

